

5. Sequential Composition, Feedback, and Traces

This section develops the transition from purely combinational composition to stateful and feedback-driven systems. The central theme is that once a circuit or machine can remember past inputs, the semantics of composition must account for time, recurrence, and fixed points. In this setting, sequential behavior is modeled by machines such as Mealy and Moore automata, by guarded recursive equations, and by categorical feedback operators captured abstractly by traced categories.

A Mealy machine produces outputs as a function of the current state and input, while a Moore machine associates outputs with states alone. Both can be viewed as finite-state realizations of a causal input–output map, and both admit composition laws that are richer than those of combinational gates. In particular, sequential composition must respect the propagation of state across time steps, so that the output at one stage may become the input to another stage in the next stage. This temporal dependency is the source of feedback, and feedback is the point at which algebraic and operational descriptions meet.

Guarded recursion provides one disciplined way to define feedback. Rather than allowing arbitrary self-reference, guarded equations require that recursive occurrences be delayed by at least one time step or otherwise protected by a causal constructor. This restriction ensures that the resulting system has a well-defined unfolding semantics and avoids ill-posed instantaneous loops. In categorical terms, traced monoidal structure abstracts the operation of feeding an output back into an input while preserving coherence laws for composition, tensoring, and iteration. The trace formalizes the idea that a feedback loop can be treated as a morphism-level operator, not merely as an ad hoc wiring pattern.

The hardware interpretation of feedback is especially important for latches and flip-flops. Standard storage elements can be built from cross-coupled NAND or NOR gates, and these constructions illustrate how sequential state emerges from combinational primitives plus feedback. Such elements also expose the distinction between synchronous and asynchronous closure. In synchronous designs, state changes are coordinated by a clock, and feedback is typically constrained to avoid race conditions and metastability. In asynchronous designs, closure under feedback is more delicate: the circuit must settle to a stable configuration without relying on a global timing discipline. The same structural loop may therefore be benign in one regime and hazardous in another.

Metastability provides a quantitative measure of this hazard. If $\varepsilon(r)$ denotes the probability of metastability as a function of a relevant parameter r such as resolution time, then the reliability of a feedback loop can be summarized by a mean time between failures (MTBF) estimate derived from $\varepsilon(r)$. In practice, the MTBF under feedback depends on the loop topology, the timing margins, and the physical characteristics of the storage element. The formal role of $\varepsilon(r)$ in this section is to connect abstract feedback semantics with an operational reliability metric: a feedback law is not only well-defined or undefined, but also more or less robust under physical implementation.

Sequential composition also interacts with minimization and symmetry. Automata minimization identifies states that are behaviorally equivalent, collapsing a machine to a smaller representative without changing its input–output behavior. This process preserves the observable symmetries of the machine while removing redundant internal structure. From the perspective of closure, minimization is a canonicalization step: it selects a normal form within an equivalence class of sequential realizations. Such normal forms are useful both for reasoning and for implementation, because they support comparison, optimization, and reuse.

The symmetry viewpoint is especially relevant when feedback introduces multiple equivalent internal encodings of the same behavior. A minimized automaton may retain only the symmetries that are semantically meaningful, while discarding accidental symmetries introduced by a particular construction. This distinction matters for formal verification, where one often wants to prove that

two implementations are equivalent up to state renaming, retiming, or other structure-preserving transformations. In this sense, sequential equivalence is not merely extensional equality of traces; it is often mediated by a chosen notion of preserved symmetry.

The section culminates in an executable artifact: an HDL implementation paired with formal equivalence checking, typically via an SMT-based backend. The workflow is to express the sequential design in hardware description language, generate a reference model or specification, and then discharge the equivalence obligation with a solver. This artifact makes the abstract discussion operational: Mealy/Moore semantics, guarded recursion, feedback, and minimization are all reflected in a machine-checkable build. When the equivalence check succeeds, it certifies that the feedback-rich implementation realizes the intended sequential behavior; when it fails, the counterexample often reveals a timing, state, or symmetry mismatch that can guide refinement.

Taken together, these ideas establish a bridge from algebraic composition to stateful systems. Sequential machines provide the operational model, traced categories provide the compositional calculus, guarded recursion provides a disciplined fixed-point principle, and formal equivalence checking provides the verification layer. The result is a framework in which feedback is not an exception to composition, but a structured and analyzable form of it.

For implementation-oriented readers, the section’s artifact can be summarized as follows: a sequential specification is encoded in HDL, a reference semantics is derived from the chosen machine model, and an SMT-backed equivalence check compares the implementation against the specification under the relevant clocking and reset assumptions. This makes the feedback loop explicit at the level of both design and proof, and it provides a concrete bridge from the categorical account of trace to the engineering practice of sequential verification.